https://portswigger.net/web-security/request-smuggling#what-is-http-request-smuggling, explain HHTP request muggling in sinhala briefly

HTTP Request Smuggling කියන්නේ මොකක්ද? (කෙටිවෙන් – සිංහලෙන්)

HTTP Request Smuggling කියන්නේ
👉 **Front-end server** (load balancer / proxy) එකක් සහ
👉 **Back-end server** (application server) එකක්
HTTP request එක **parse** කරන විදිහ වෙනස් වීම exploit කරලා,
එක **request** එකක් තුළින් **server** දෙකට **request** දෙකක් වගේ තේරෙන්න කරන **attack** එකක්.

සරලව කියනවා නම්

- Browser → Front-end → Back-end
- Front-end එක request එක **එක් විදිහකට** තේරුම් ගන්නවා
- Back-end එක ඒක **වෙනස් විදිහකට** තේරුම් ගන්නවා
- Attacker ඒ difference එක exploit කරලා **extra request** එකක් **"smuggle"** කරනවා

ඇයි මෙහෙම වෙන්නේ?

HTTP headers දෙක නිසා:

- `Content-Length`
- `Transfer-Encoding: chunked`

Server දෙකම මේ දෙක **priority** දෙන විදිහ වෙනස් නම්,
request boundary වැරදිලා → smuggling වෙයි.

Common Types (නම දැනගෙන ඉන්න හොඳයි)

- **CL.TE** – Front-end uses Content-Length, Back-end uses Transfer-Encoding
- **TE.CL** – Front-end uses Transfer-Encoding, Back-end uses Content-Length

---

Attackerට මොනවද කරන්න පුළුවන්?

- වෙන userගේ request hijack කරන්න
- Admin request inject කරන්න

- Cache poisoning
- Session hijacking
- Access control bypass

කෙටි example idea

Attacker request එකක් send කරනවා
Front-end හිතන්නේ:

> "මේ request එක මෙතනින් ඉවරයි"

Back-end හිතන්නේ:

> "නෑ, තව request එකක් තියෙනවා"

👉 ඒ extra request එක attackerගේ control එකේ.

Summary (exam / interview style)

> **HTTP Request Smuggling** යනු multiple servers HTTP request boundaries වෙනස් ලෙස interpret කරන විට, attackerට hidden HTTP requests inject කරගැනීමට ඉඩ දෙන vulnerability එකකි.

ඕනේ නම්
➡️ **diagram එකක්,**
➡️ **Burp Suite lab එක explain කරලා,**
➡️ **CL.TE / TE.CL example raw requests**
සිංහලෙන් step-by-step කියලා දෙන්න පුළුවන්.

↪ ### ඇයි මෙහෙම වෙන්නේ? HTTP headers දෙක නිසා: * `Content-Length` * `Transfer-Encoding: chunked` Server දෙකම මේ දෙක **priority දෙන විදිහ වෙනස්** නම්, request boundary වැරදිලා → smuggling වෙයි. * * * ### Common Types (නම දැනගෙන ඉන්න හොඳයි) * **CL.TE** – Front-end...

expalin more

හරි 👍
එහෙනම් **Content-Length** සහ **Transfer-Encoding** headers දෙක නිසා HTTP Request Smuggling වෙන්නේ **කොහොමද කියලා deep but simple** විදිහට explain කරලා දෙන්නම්.

1️⃣ HTTP Request එක server එකට කොහොමද "අවසානය" තීරණය වෙන්නේ?

HTTP server එකකට **මොකක්ද request එකේ body** එක ඉවර වෙන්නේ කියලා දැනගන්න headers භාවිතා කරනවා.

ඒකට main methods දෙකක් තියෙනවා 👇

🔹 Method 1: `Content-Length`

```http
POST /login HTTP/1.1
Host: example.com
Content-Length: 20

username=admin&pw=1
```

Copy code

👉 මෙහි server එක කියන්නේ:

> "Body එක **20 bytes** තියෙනවා, ඒක ඉවර වුණාම request එක complete."

🔹 Method 2: `Transfer-Encoding: chunked`

```http
POST /login HTTP/1.1
Host: example.com
Transfer-Encoding: chunked

4
test
0
```

Copy code

👉 මෙහි:

- `4` → bytes ගණන
- `test` → data
- `0` → **request body** ඉවරයි කියන signal

2️⃣ ප්‍රශ්නය කොතනද?

🔥 **HTTP standard** එක කියනවා:

> `Transfer-Encoding` තියෙනවා නම් → `Content-Length` ignore කරන්න ඕනේ

නමුත්...

😬 **සියලු server implementations එකම විදිහට behave කරන්නෙ නෑ**

3️⃣ Front-end vs Back-end (මෙතනයි vulnerability එක)

සාමාන්‍ය web architecture එකක් බලමු 👇

```pgsql
Browser
   ↓
Front-end server (Proxy / Load Balancer)
   ↓
Back-end server (Application)
```

Copy code

👉 **Request එක parse කරන්නේ server දෙකම තමන්ගේ logic එකෙන්**

4️⃣ CL.TE attack (Content-Length → Transfer-Encoding)

💥 Scenario

- **Front-end** → `Content-Length` trust කරනවා
- **Back-end** → `Transfer-Encoding` trust කරනවා

🧪 Attacker request එක

```http
POST / HTTP/1.1
Host: victim.com
Content-Length: 13
Transfer-Encoding: chunked

0

GET /admin HTTP/1.1
Host: victim.com
```

Copy code

🧠 Front-end server හිතන්නේ:

- `Content-Length: 13`
- Body එක = `0\r\n\r\n`
- 👉 **Request එක මෙතනින් ඉවරයි** ✅
- ඉතුරු data back-endට pass කරනවා

🧠 Back-end server හිතන්නේ:

- `Transfer-Encoding: chunked`
- `0` → chunk end
- **ඊළඟ data එක new HTTP request එකක්!**

```http
GET /admin HTTP/1.1
Host: victim.com
```

Copy code

🔥 **Admin request එක attacker "smuggle" කරලා** 😈

5️⃣ TE.CL attack (Transfer-Encoding → Content-Length)

💥 Scenario

- **Front-end →** `Transfer-Encoding`
- **Back-end →** `Content-Length`

🧪 Attacker request

```http
POST / HTTP/1.1
Host: victim.com
Transfer-Encoding: chunked
Content-Length: 4

5
HELLO
0

GET /admin HTTP/1.1
Host: victim.com
```

Copy code

🗨️ **Front-end:**

- Chunked encoding correctly parse කරනවා
- HELLO + end
- Remaining data next request ලෙස send

🗨️ **Back-end:**

- `Content-Length: 4`
- Body = `5\r\nH`
- ඉතුරු data **request boundary** එකෙන් එළියට

👉 Request confusion 😵

6️⃣ Request Smuggling නිසා attackerට ලැබෙන power

✔️ Other userගේ request hijack
✔️ Admin endpoints call කරන්න
✔️ Cache poisoning
✔️ Authentication bypass
✔️ WAF bypass

7️⃣ Simple one-line exam answer

> **HTTP Request Smuggling** occurs when front-end and back-end servers interpret request boundaries differently due to inconsistent handling of `Content-Length` and `Transfer-Encoding` headers.

ඊළඟට කැමති නම් 👇

- 🔹 Burp Suite lab එක step-by-step
- 🔹 Raw requests practice
- 🔹 Detection techniques
- 🔹 Real-world exploitation flow

කියන්න 👍

how it do in real world

හොඳ ප්‍රශ්නයක් 👍

“Real world එකේ HTTP Request Smuggling කොහොමද actually කරනේ?” කියලා **attacker workflow** **එකක්** විදිහට (conceptual + practical) explain කරලා දෙන්නම්.

⚠️ මෙක **learning / defense** අරමුණට.

1️⃣ Real-world setup එක (බොහෝ වෙබ් apps වල)

```pgsql
User / Attacker
      ↓
CDN / WAF / Reverse Proxy (Front-end)
      ↓
Application Server (Back-end)
```

Copy code

👉 Examples:

- Front-end: Cloudflare, Akamai, Nginx, HAProxy
- Back-end: Apache, Tomcat, Node.js, Spring, PHP

Vulnerability **එන්නේ** 👉

Front-end එක request parse කරන විදිහ ≠ Back-end එක parse කරන විදිහ

2️⃣ Attacker එකක් real world එකේ මුලින්ම කරන්නේ මොකක්ද?

🔍 Step 1: Application behavior observe කරනවා

- Login
- Search
- Cart
- API endpoints
- POST requests

👉 Special attention:

- HTTP/1.1 usage
- POST requests with body
- Load balancer / proxy present ද?

3️⃣ Step 2: Smuggling possible ද කියලා test කරනවා

Attacker **normal request break** නොවී server දෙකම headers interpret කරන විදිහ test කරනවා.

Example tests (idea level):

- `Content-Length` + `Transfer-Encoding` එකටම add කරනවා
- Response delay / weird behavior observe කරනවා
- 400 vs 200 responses

👉 Symptoms:

- Hanging requests
- Response mixing
- Other user content leak
- Random redirects

4️⃣ Step 3: Find parsing mismatch (this is key 🔑)

Attacker identify කරනවා:

Front-end trusts	Back-end trusts	Type
Content-Length	Transfer-Encoding	CL.TE
Transfer-Encoding	Content-Length	TE.CL

👉 මේක identify කරන්නේ **trial + error + timing analysis** වලින්.

5️⃣ Step 4: Smuggled request inject කරනවා

🔥 මෙතන attackerගේ main goal එක:

> **Back-end server එකට hidden request එකක් execute කරවන්න**

Examples:

- `/admin`
- `/api/users`
- `/logout`
- `/delete-account`

👉 Front-end එක හිතන්නේ:

> "Request එක safe"

👉 Back-end එක හිතන්නේ:

> "New valid request"

6️⃣ Step 5: Real-world exploitation scenarios

🧨 Scenario 1: Admin access

- Smuggled request = `GET /admin`
- Back-end session reuse වෙනවා
- Admin-only page exposed

🧨 Scenario 2: User request hijacking

- Attacker smuggles partial request
- Next real user request → attacker payload completes it
- User unknowingly attacker request execute කරනවා

🧨 Scenario 3: Cache poisoning (very common)

- Smuggled request poisons cache
- All users get malicious response
- Stored XSS / redirect attacks

🧨 Scenario 4: Authentication bypass

- Login flow break වෙනවා
- Session fixation
- Logout forced on other users

7️⃣ Why this works in production (important!)

✔️ Different vendors
✔️ Legacy servers
✔️ Misconfigured WAFs
✔️ HTTP/1.1 ambiguity
✔️ Backward compatibility issues

👉 **Even big companies had this bug** (PayPal, Amazon, etc.)

8️⃣ Why HTTPS / WAF doesn't stop it?

❌ HTTPS only encrypts traffic
❌ WAF often parses request **once**
❌ Back-end parses again

👉 Vulnerability is **between servers**, not browser ↔ server

9️⃣ Blue Team / Dev perspective (how they fix)

✔ Disable HTTP/1.1 ambiguity
✔ Reject requests with both headers
✔ Normalize requests at edge
✔ Use same server software front & back
✔ Strict RFC compliance

🔑 One-line real-world summary

> In the real world, HTTP Request Smuggling is performed by exploiting differences in how front-end and back-end servers interpret HTTP request boundaries, allowing attackers to inject hidden requests that the front-end never sees.

ඊළඟට කැමති නම් 👇

- 🔹 **Burp Suite** real lab flow
- 🔹 **Detection checklist (pentesting)**
- 🔹 **How bug bounty hunters find this**
- 🔹 **Defense guide for developers**

කියන්න 👍

how measure content-length ?

හොඳ ප්‍රශ්නයක් 👍
Content-Length "measure" (ගණනය) කරන්නේ කොහොමද? කියලා **real HTTP level** එකේ පැහැදිලිව explain කරන්නම්.

1️⃣ Content-Length කියන්නේ මොකක්ද?

👉 **HTTP request body** එකේ **bytes** ගණන
(Characters ගණන නෙවෙයි – **bytes**)

```http
```

```
Content-Length: <number of bytes>
```

Copy code

Server එක කියන්නේ:

> "Body එකේ bytes මෙච්චරයි, ඒක ඉවර වුණාම request එක complete."

2️⃣ Important rule (මතක තියාගන්න 🔑)

> **Content-Length = request BODY එකේ bytes only**
> ❌ Headers count කරන්නේ නෑ
> ❌ Blank line (`\r\n\r\n`) count කරන්නේ නෑ

3️⃣ Simple example (ASCII text)

```http
POST /login HTTP/1.1
Host: example.com
Content-Length: 11

hello=world
```

Copy code

Body:

```ini
hello=world
```

Copy code

Byte count:

```powershell
h e l l o = w o r l d
1 1 1 1 1 1 1 1 1 1 1
```

Copy code

👉 **Total = 11 bytes** ✅

4️⃣ Newline characters count වෙනවාද?

ඔව් ⚠️ `\r\n` = 2 bytes

Example body:

```bash
test
ok
```

Actual bytes:

```css
t e s t \r \n o k
1 1 1 1  2     1 1
```

👉 **Total = 8 bytes**

5️⃣ Smuggling attacks වලදී content-length measure කරන විදිහ

Attacker intentionally **mismatch** කරනවා.

Example:

```http
POST / HTTP/1.1
Host: victim.com
Content-Length: 5

0

GET /admin HTTP/1.1
```

What attacker counts as body:

```css
0\r\n\r\n
```

Byte count:

- `0` → 1
- `\r\n` → 2
- `\r\n` → 2

  👉 **Total = 5 bytes**

🔥 Front-end:

> "Body complete"

🔥 Back-end:

> "Next request starts here"

6️⃣ UTF-8 characters වලදී (VERY IMPORTANT)

ASCII නොවන characters **bytes** එකකට වඩා ගන්නවා

Character	Bytes
a	1
1	1
=	1
අ	3
😊	4

Example:

```ini
name=අ
```

Bytes:

- n a m e = → 5
- අ → 3

  👉 **Total = 8 bytes**

⚠️ Character count ≠ byte count

7️⃣ How tools calculate Content-Length

🔹 Burp Suite

- Automatically byte-accurate
- Raw tab → exact bytes

🔹 curl

- Auto sets Content-Length
- But raw mode manual editing possible

🔹 **Python (requests)**

- Calculates bytes after encoding

❌ Forgetting `\r\n`
❌ Copy–paste from editor (adds hidden chars)
❌ UTF-8 chars miscount

9️⃣ Interview-style one-liner

> **Content-Length is the number of bytes in the HTTP request body, not characters, and includes all CRLF sequences.**

🔑 Smuggling context summary

- Front-end → trusts Content-Length
- Attacker → carefully counts bytes
- Boundary mismatch → request smuggling

ඊළඟට කැමති නම් 👇

- 🔹 **Burp Suite එකෙන් byte-by-byte calculate කරන විදිහ**
- 🔹 **TE vs CL difference with real bytes**
- 🔹 **Practice exercise (you count, I verify)**

කියන්න 👍

how this work ~Lab: HTTP request smuggling, basic TE.CL vulnerability
PRACTITIONER

LAB
Not solved
This lab involves a front-end and back-end server, and the back-end server doesn't support chunked encoding. The

front-end server rejects requests that aren't using the GET or POST method.

To solve the lab, smuggle a request to the back-end server, so that the next request processed by the back-end server appears to use the method GPOST.

Note
Although the lab supports HTTP/2, the intended solution requires techniques that are only possible in HTTP/1. You can manually switch protocols in Burp Repeater from the Request attributes section of the Inspector panel.

Tip
Manually fixing the length fields in request smuggling attacks can be tricky. Our HTTP Request Smuggler Burp extension was designed to help. You can install it via the BApp Store.

ACCESS THE LAB
Solution
In Burp Suite, go to the Repeater menu and ensure that the "Update Content-Length" option is unchecked.

Using Burp Repeater, issue the following request twice:

```
POST / HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-length: 4
Transfer-Encoding: chunked

5c
GPOST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 15

x=1
0~
```

හරි 👍

මේ **PortSwigger – Basic TE.CL HTTP Request Smuggling lab** එක ඇත්තටම කොහොමද **work** වෙන්නේ කියලා **step-by-step, byte level** එකටත් සිංහලෙන් explain කරන්නම්.

🔴 Lab situation එක (ඉතා වැදගත්)

Architecture

```pgsql
Browser
   ↓
Front-end server  (supports chunked, enforces method check)
   ↓
Back-end server   (❌ DOES NOT support chunked encoding)
```

Lab rules

- Front-end server:
  - `Transfer-Encoding: chunked` **trust** කරනවා
  - GET / POST විතරයි allow
- Back-end server:
  - ❌ chunked encoding support නෑ
  - `Content-Length` trust කරනවා

👉 මේක **TE.CL vulnerability** එකක්.

1️⃣ Attackerගේ main goal (lab objective)

> **Back-end server** එකට **process** වෙන **next request** එකේ **HTTP method** එක `GPOST` ලෙස **appear** කරවන්න

⚠️ `GPOST` කියන්නේ invalid method එකක්

👉 Back-end request parsing **desync** වුණා කියලා prove කරන්න.

2️⃣ Attack request එක (ඔයා send කරන එක)

```http
POST / HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-length: 4
Transfer-Encoding: chunked

5c
GPOST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 15

x=1
0~
```

⚠️ **Burp Repeater → Update Content-Length = OFF**
(නැත්නම් Burp auto fix කරලා attack එක fail වෙනවා)

3️⃣ Front-end server එක request එක කොහොමද තේරුම් ගන්නේ? (TE)

Front-end **Transfer-Encoding: chunked** trust කරනවා.

Chunk size line

```r
5c
```

👉 `5c` (hex) = **92 bytes**

Front-end කියන්නේ:

> "ඊළඟ bytes 92ක් body එක"

Front-end body කියලා ගන්න data

```http
GPOST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 15

x=1
```

👉 මේ data එක **92 bytes** ඇතුළත තියෙනවා

ඊළඟ line:

```
0~
```

👉 `0` = chunk end
👉 Front-end:

> "Request body complete ✅"

🔥 **Front-end එකට මේක valid POST request එකක්**

4️⃣ Back-end server එක request එක කොහොමද තේරුම් ගන්නේ? (CL)

❌ Back-end **chunked encoding support** නෑ

👉 `Transfer-Encoding` ignore කරනවා

👉 `Content-Length: 4` trust කරනවා

Back-end body = first 4 bytes only

Body start:

```
5c\r\n
```

👉 Bytes:

- `5` → 1
- `c` → 1
- `\r` → 1
- `\n` → 1

  👉 **Total = 4 bytes**

🔥 Back-end:

> "Body complete. Request finished."

5️⃣ Now the MAGIC (smuggling happens here 🔥)

Back-end server එකට ඊළඟ **data** new HTTP request එකක් වගේ පේනවා 👇

```http
GPOST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 15

x=1
```

👉 Back-end හිතන්නේ:

> “අලුත් request එකක් ආවා”

👉 Method = **GPOST** ❌

6️⃣ Lab success condition

Lab checks:

> “Does back-end process a request with method `GPOST` ?”

✔️ Yes → **Lab SOLVED**

7️⃣ Why request send twice?

PortSwigger reason:

- First request → poison back-end connection
- Second request → smuggled request gets executed

👉 **Persistent connection desync** trigger වෙන්න.

8️⃣ One-line core reason (exam / interview gold 🥇)

> This TE.CL request smuggling attack works because the front-end server uses `Transfer-Encoding` to determine the request boundary, while the back-end server ignores chunked encoding and uses `Content-Length`, causing request desynchronization.

9️⃣ Very short flow diagram

arduino Copy code

```arduino
Front-end sees:
POST + chunked → OK

Back-end sees:
POST (CL=4) → ends early
↓
GPOST / HTTP/1.1 → new request
```

🔑 What you REALLY learned here

✔️ Hex chunk sizes
✔️ Byte-accurate Content-Length
✔️ Front-end vs Back-end mismatch
✔️ Real desync exploitation
✔️ Why WAFs fail

ඔයාට ඕනේ නම් 👇

- 🔹 Same lab **diagram** එකක්
- 🔹 **Byte-by-byte count table**
- 🔹 **How to detect TE.CL in pentests**
- 🔹 **How to fix this as a developer**

කියන්න 👍